Integrate once, merge exactly: accumulator-based data reduction for mapping and time-resolved scattering experiments

Jérôme Kieffer, Edgar Gutiérrez-Fernández, Manfred Burghammer,
Jonathan Wright, Loïc Huder and Thomas Vincent

European Synchrotron Radiation Facility (ESRF), Grenoble, France

November 3, 2026

Abstract

Schubert and Gertz [1] derive single-pass, numerically stable and mergeable formulas for the weighted mean and (co-)variance, where the same weights ω_i are used for the mean and for the sum of deviation products. The `azimuthal` error model of pyFAI weights the mean with ω_i but the deviations with ω_i^2. This note re-derives Section 3 of [1] in this setting. A cross-term appears, which vanishes in [1]: an exact merge of two partitions requires one additional accumulator per variable, $D_P^V = \sum_{i \in P} \omega_i^2 (v_i - \widehat{v}_P)$. Equation numbers of the form (SGn) refer to equation n of [1]. The motivation is to store pre-integrated accumulators instead of raw images for maps and time series, which saves about one order of magnitude in storage. A simulation validates the approach, and two experimental examples are presented: the decrease of the spottiness of a powder pattern with the number of merged frames (ID11, ESRF), and the measurement of texture and orientation in a Kevlar fibre (ID13, ESRF).

1 Motivation: saving storage for maps and time series

In a mapping or time-resolved experiment, all frames are first integrated. The map built from these results is used to define regions with equivalent signal, and the frames of each region are then combined to improve the statistics. Two workflows are possible:

1. The raw frames are kept. For each region, they are averaged and integrated a second time.
2. Each frame is integrated once, and the per-bin accumulators are stored in addition to (q, I, σ). For each region, the pre-integrated accumulators of the selected frames are merged, without going back to the raw data.

As an example, consider 10^6 frames from an Eiger 4M detector (2167×2070 pixels, 32-bit integers), integrated in 2D on 4000 radial $\times$ 8 azimuthal bins. We assume a compression ratio of 8 for the raw integer data and of 3 for floating-point data. The accumulators per bin are the weighted sum of signal X_P, the sum of weights Ω_P, the sum of squared weights $\Omega_P^{(2)}$, the pixel count, the sum of squared deviations XX_P and the new accumulator D_P^X introduced in Section 3.

Table 1 shows that storing the accumulators saves an order of magnitude compared to keeping the raw frames. Compared to storing only (I, σ) (85 GB), the overhead is 43–171 GB, i.e. 2–8 % of the raw data volume. Ω_P, $\Omega_P^{(2)}$ and the count only depend on the geometry, the mask and the normalization (solid angle, polarization, flat-field). When those are static, they can be stored once, a per-frame monitor being a scalar factor ($\Omega_P \propto m$, $\Omega_P^{(2)} \propto m^2$). Dynamic masks or sigma-clipping make them frame-dependent. The noisy accumulators XX_P and D_P^X may also compress less than assumed. The second pass is cheaper as well: about 9 times less data is read, and the merge of 32 000 bins replaces the integration of 4.5 Mpixels per frame.

Stored data	per frame	10^6 frames	gain
Workflow 1: raw frames (32-bit) + I, σ (float32)	2.33 MB	2.33 TB	1
Workflow 2: 6 accumulators (float32)	256 kB	256 GB	9.1
Workflow 2: 6 accumulators (float64)	512 kB	512 GB	4.5
Workflow 2: X_P, XX_P, D_P^X (float32); Ω_P, $\Omega_P^{(2)}$, count shared	128 kB	128 GB	18.2

Table 1: Storage needed by both workflows for 10^6 Eiger 4M frames integrated on 4000×8 bins.

Workflow 2 requires an exact merge of the accumulators with the `azimuthal` error model, which is the subject of this note.

2 Why squared weights in pyFAI

In pyFAI, each pixel i provides a raw signal s_i and a normalization n_i (solid angle, polarization, flat-field, ...), and contributes to a bin with a pixel-splitting coefficient c_i. With $x_i = s_i/n_i$ and $\omega_i = c_i n_i$, the integrated intensity of a bin P is the weighted mean

$$\widehat{x}_P = \frac{\sum_{i\in P} c_i s_i}{\sum_{i\in P} c_i n_i} = \frac{\sum_{i\in P} \omega_i x_i}{\sum_{i\in P} \omega_i}. \tag{1}$$

Neglecting the uncertainty of the denominator, its variance is $\sum_{i\in P} c_i^2\, \sigma^2(s_i)/\Omega_P^2$. The `poisson` error model propagates a known $\sigma^2(s_i)$, while the `azimuthal` model estimates it from the dispersion within the bin: $\sigma^2(s_i) \approx n_i^2 (x_i - \widehat{x}_P)^2$. Both models then share the same form:

$$\sigma^2(\widehat{x}_P) \approx \frac{1}{\Omega_P^2} \sum_{i\in P} \omega_i^2 (x_i - \widehat{x}_P)^2 = \frac{XX_P}{\Omega_P^2}, \tag{2}$$

so that pyFAI reports sem $= \sqrt{XX_P}/\Omega_P$ and std $= \sqrt{XX_P/\Omega_P^{(2)}}$. The mean is weighted by ω_i, the sum of squared deviations by ω_i^2.

3 Notation

Table 2 extends Table 1 of [1]. The only changes are the sum of squared weights $\Omega_P^{(2)}$, the ω^2 weighting of the deviation products VW_P, and the new first-order deviation sums D_P^V.

Symbol	Meaning	pyFAI ($V = W = X$)
A, B, P	partitions of the data set	bins, sub-bins
$A \sqcup B$	union of disjoint partitions A and B	
$A \sqcup \{b\}$	partition A plus a single point $b \notin A$	
$X = \{x_i\}$, Y, V, W	variables	$x_i = s_i/n_i$
$\Omega = \{\omega_i\}$	weights	$\omega_i = c_i n_i$
$V_P = \sum_{i\in P} \omega_i v_i$	weighted sum of V	`sum_signal`
$\Omega_P = \sum_{i\in P} \omega_i$	sum of weights	`sum_normalization`
$\Omega_P^{(2)} = \sum_{i\in P} \omega_i^2$	sum of squared weights	`sum_normalization2`
$\widehat{v}_P = \frac{1}{\Omega_P} V_P$	weighted mean of V	`intensity`
$VW_P = \sum_{i\in P} \omega_i^2 (v_i - \widehat{v}_P)(w_i - \widehat{w}_P)$	ω^2-weighted sum of deviation products	`sum_variance`
$D_P^V = \sum_{i\in P} \omega_i^2 (v_i - \widehat{v}_P)$	ω^2-weighted sum of deviations	*new accumulator*

Table 2: Notation, adapted from Table 1 of [1].

In [1], $VW_P = \sum_{i\in P} \omega_i(v_i - \widehat{v}_P)(w_i - \widehat{w}_P)$, and the analogue of D_P^V, namely $\sum_{i\in P} \omega_i(v_i - \widehat{v}_P)$, is identically zero by definition of the mean. With squared weights,

$$D_P^V = \sum_{i\in P} \omega_i^2 v_i - \Omega_P^{(2)}\, \widehat{v}_P = \Omega_P^{(2)} \left(\widehat{v}_P^{(\omega^2)} - \widehat{v}_P\right), \tag{3}$$

where $\widehat{v}_P^{(\omega^2)}$ is the ω^2-weighted mean. D_P^V vanishes when both means coincide, in particular when all weights within P are equal, but not in general.

4 Updateable weighted (co-)variance

4.1 Mean and "textbook" form

The sums V_P, Ω_P and $\Omega_P^{(2)}$ are additive over disjoint partitions, (SG14)–(SG15):

$$V_{A\sqcup B} = V_A + V_B, \qquad \Omega_{A\sqcup B} = \Omega_A + \Omega_B, \qquad \Omega^{(2)}_{A\sqcup B} = \Omega^{(2)}_A + \Omega^{(2)}_B. \tag{4}$$

The update of the mean is unchanged, (SG16)–(SG17):

$$\widehat{v}_{A\sqcup B} = \frac{\Omega_A}{\Omega_{A\sqcup B}}\widehat{v}_A + \frac{\Omega_B}{\Omega_{A\sqcup B}}\widehat{v}_B = \widehat{v}_A + \frac{\Omega_B}{\Omega_{A\sqcup B}}\left(\widehat{v}_B - \widehat{v}_A\right). \tag{5}$$

Expanding the product as in (SG18)–(SG19) gives the analogue of the König–Huygens formula:

$$VW_P = \sum_{i\in P} \omega_i^2 v_i w_i - \widehat{w}_P \sum_{i\in P} \omega_i^2 v_i - \widehat{v}_P \sum_{i\in P} \omega_i^2 w_i + \Omega_P^{(2)}\, \widehat{v}_P\, \widehat{w}_P. \tag{6}$$

Unlike (SG19), the last three terms do not collapse into a single product, as $\widehat{v}_P$ is not the ω^2-weighted mean. Like the "textbook" algorithm, Equation (6) only involves plain sums, which are trivially mergeable, but it suffers from catastrophic cancellation when $VW_P \ll \Omega_P^{(2)}\, \widehat{v}_P\, \widehat{w}_P$. It should not be used.

4.2 Derivation of the general form

Section 3.1 of [1] proves (SG21) through an identity derived "backwards". With squared weights, a direct decomposition is simpler. Let $\delta_A^v = \widehat{v}_A - \widehat{v}_{A\sqcup B}$ be the shift of the mean of A with respect to the mean of the union. Writing $v_i - \widehat{v}_{A\sqcup B} = (v_i - \widehat{v}_A) + \delta_A^v$ and similarly for w:

$$\begin{aligned}\sum_{i\in A} \omega_i^2 (v_i - \widehat{v}_{A\sqcup B})(w_i - \widehat{w}_{A\sqcup B}) &= \sum_{i\in A} \omega_i^2 \left[(v_i - \widehat{v}_A) + \delta_A^v\right]\left[(w_i - \widehat{w}_A) + \delta_A^w\right] \\ &= VW_A + \delta_A^w\, D_A^V + \delta_A^v\, D_A^W + \Omega_A^{(2)}\, \delta_A^v\, \delta_A^w. \end{aligned} \tag{7}$$

In [1], the two middle terms vanish. Adding the same expression for B gives the "parallel axis" form:

$$VW_{A\sqcup B} = VW_A + VW_B + \sum_{P\in\{A,B\}} \left(\Omega_P^{(2)}\, \delta_P^v\, \delta_P^w + \delta_P^w\, D_P^V + \delta_P^v\, D_P^W\right). \tag{8}$$

From Equation (5), with $\Delta v = \widehat{v}_A - \widehat{v}_B$:

$$\delta_A^v = \frac{\Omega_B}{\Omega_{A\sqcup B}}\, \Delta v, \qquad \delta_B^v = -\frac{\Omega_A}{\Omega_{A\sqcup B}}\, \Delta v. \tag{9}$$

The same decomposition applied to $D^V_{A\sqcup B} = \sum_{i\in A\sqcup B} \omega_i^2 (v_i - \widehat{v}_{A\sqcup B})$ gives $D^V_{A\sqcup B} = D_A^V + D_B^V + \Omega_A^{(2)}\, \delta_A^v + \Omega_B^{(2)}\, \delta_B^v$, hence:

Proposition 1 (General form, analogue of (SG21))**.** *For disjoint partitions A and B, with $\Delta v = \widehat{v}_A - \widehat{v}_B$ and $\Delta w = \widehat{w}_A - \widehat{w}_B$:*

$$VW_{A\sqcup B} = VW_A + VW_B + \frac{\Omega_A^{(2)}\,\Omega_B^2 + \Omega_B^{(2)}\,\Omega_A^2}{\Omega_{A\sqcup B}^2}\,\Delta v\,\Delta w + \frac{\left(\Omega_B D_A^V - \Omega_A D_B^V\right)\Delta w + \left(\Omega_B D_A^W - \Omega_A D_B^W\right)\Delta v}{\Omega_{A\sqcup B}}, \tag{10}$$

$$D_{A\sqcup B}^V = D_A^V + D_B^V + \frac{\Omega_A^{(2)}\,\Omega_B - \Omega_B^{(2)}\,\Omega_A}{\Omega_{A\sqcup B}}\,\Delta v. \tag{11}$$

Variance. For $V = W = X$, the case used by pyFAI:

$$XX_{A\sqcup B} = XX_A + XX_B + \frac{\Omega_A^{(2)}\,\Omega_B^2 + \Omega_B^{(2)}\,\Omega_A^2}{\Omega_{A\sqcup B}^2}\,\Delta x^2 + 2\,\frac{\Omega_B D_A^X - \Omega_A D_B^X}{\Omega_{A\sqcup B}}\,\Delta x, \tag{12}$$

$$D_{A\sqcup B}^X = D_A^X + D_B^X + \frac{\Omega_A^{(2)}\,\Omega_B - \Omega_B^{(2)}\,\Omega_A}{\Omega_{A\sqcup B}}\,\Delta x. \tag{13}$$

Equation (12) is symmetric in A and B: unlike the current pyFAI implementation, the partitions do not need to be ordered by weight.

Sufficient statistics. In [1], the four values $(VW_P, \widehat{v}_P, \widehat{w}_P, \Omega_P)$ are sufficient statistics for merging. With squared weights, $(VW_P, \widehat{v}_P, \widehat{w}_P, \Omega_P, \Omega_P^{(2)}, D_P^V, D_P^W)$ are needed. For the variance alone, this means five accumulators per bin: X_P, Ω_P, $\Omega_P^{(2)}$, XX_P and D_P^X. D_P^X cannot be dropped: two partitions may share the same $(X_P, \Omega_P, \Omega_P^{(2)}, XX_P)$ but have different D_P^X, and therefore give different XX once merged with the same third partition. No formula based on these four accumulators can be exact.

Reduction to [1]. If $D_A^V = D_B^V = D_A^W = D_B^W = 0$ and $\Omega^{(2)}$ is replaced by Ω (same weights for mean and deviations), the coefficient of $\Delta v\,\Delta w$ becomes $(\Omega_A\Omega_B^2 + \Omega_B\Omega_A^2)/\Omega_{A\sqcup B}^2 = \Omega_A\Omega_B/\Omega_{A\sqcup B}$, and Equation (10) reduces to (SG21).

Form using sums. As in (SG22), the means can be replaced by the sums, using $\Delta v = (\Omega_B V_A - \Omega_A V_B)/(\Omega_A\Omega_B)$:

$$VW_{A\sqcup B} = VW_A + VW_B + \frac{\left(\Omega_A^{(2)}\Omega_B^2 + \Omega_B^{(2)}\Omega_A^2\right)(\Omega_B V_A - \Omega_A V_B)\,(\Omega_B W_A - \Omega_A W_B)}{\Omega_A^2\,\Omega_B^2\,\Omega_{A\sqcup B}^2} + \frac{\left(\Omega_B D_A^V - \Omega_A D_B^V\right)(\Omega_B W_A - \Omega_A W_B) + \left(\Omega_B D_A^W - \Omega_A D_B^W\right)(\Omega_B V_A - \Omega_A V_B)}{\Omega_A\,\Omega_B\,\Omega_{A\sqcup B}}. \tag{14}$$

This is the form to use with the accumulators of pyFAI, which stores sums rather than means.

4.3 Specializations to a single point

Section 3.2 of [1] specializes the general form to a partition $B = \{b\}$ made of a single point. Its statistics are $V_b = \omega_b v_b$, $\Omega_b = \omega_b$, $\Omega_b^{(2)} = \omega_b^2$, $VW_b = 0$ and also $D_b^V = 0$. With $\Delta v = \widehat{v}_A - v_b$:

$$VW_{A\sqcup\{b\}} = VW_A + \omega_b^2\,\frac{\Omega_A^{(2)} + \Omega_A^2}{\Omega_{A\sqcup\{b\}}^2}\,\Delta v\,\Delta w + \omega_b\,\frac{D_A^V\,\Delta w + D_A^W\,\Delta v}{\Omega_{A\sqcup\{b\}}}, \tag{15}$$

$$D_{A\sqcup\{b\}}^V = D_A^V + \omega_b\,\frac{\Omega_A^{(2)} - \omega_b\,\Omega_A}{\Omega_{A\sqcup\{b\}}}\,\Delta v. \tag{16}$$

The term in D_A^V remains: even an online algorithm, which adds one point at a time, has to carry D_A^V. Only the contribution of the new point vanishes.

Analogue of Welford's update (SG27). Welford [2], in the form given by Knuth, updates the sum of squared deviations with the product of the deviations to the old and new means. For $V = W = X$, using $\widehat{x}_{A\sqcup\{b\}} - x_b = \frac{\Omega_A}{\Omega_{A\sqcup\{b\}}}(\widehat{x}_A - x_b)$, Equation (15) reads

$$XX_{A\sqcup\{b\}} = XX_A + \omega_b^2 \frac{\Omega_A^{(2)} + \Omega_A^2}{\Omega_A\,\Omega_{A\sqcup\{b\}}} \left(\widehat{x}_{A\sqcup\{b\}} - x_b\right)(\widehat{x}_A - x_b) + 2\,\omega_b \frac{D_A^X}{\Omega_{A\sqcup\{b\}}}(\widehat{x}_A - x_b)\,. \qquad (17)$$

The update currently implemented in pyFAI is (SG27) with ω_b^2 in place of ω_b: $XX_{A\sqcup\{b\}} \approx XX_A + \omega_b^2(\widehat{x}_{A\sqcup\{b\}} - x_b)(\widehat{x}_A - x_b)$. Compared to Equation (17), it misses the factor $(\Omega_A^{(2)} + \Omega_A^2)/(\Omega_A\Omega_{A\sqcup\{b\}})$ and the D_A^X term. Both corrections vanish when all the weights of $A \sqcup \{b\}$ are equal: $\Omega_A^{(2)} = \omega_b\Omega_A$ and $D_A^X = 0$. This is why the current formula is accurate for nearly uniform weights, and in particular for unweighted data.

Analogue of Youngs and Cramer's update (SG28). Youngs and Cramer [3] use the running sum instead of the running mean. Using the sum X_A instead of the mean, with $\widehat{x}_A - x_b = (X_A - \Omega_A x_b)/\Omega_A$:

$$XX_{A\sqcup\{b\}} = XX_A + \omega_b^2 \frac{\Omega_A^{(2)} + \Omega_A^2}{\Omega_A^2\,\Omega_{A\sqcup\{b\}}^2}(X_A - \Omega_A x_b)^2 + 2\,\omega_b \frac{D_A^X}{\Omega_A\,\Omega_{A\sqcup\{b\}}}(X_A - \Omega_A x_b)\,. \qquad (18)$$

As in [1], the division comes last and only XX and D^X depend on it. This is the preferred form for pipelined CPUs and GPUs.

4.4 Numerical remarks

- D_P^V could be replaced by the plain sum $\sum_{i\in P}\omega_i^2 v_i$, which is trivially mergeable (Equation (3)). However, recovering D_P^V from it involves the difference of two large, close numbers. The centered quantity D_P^V is preferred for the same reason as VW_P is preferred to $\sum \omega_i^2 v_i w_i$.
- XX_P and $\Omega_P^{(2)}$ are sums of positive terms and are insensitive to cancellation: single precision is sufficient for an uncertainty. D_P^X is a correction term, also accurate enough in single precision. Conversely, $\Delta x = \widehat{x}_A - \widehat{x}_B$ is a difference of close numbers: X_P and Ω_P deserve the highest available precision (double-word arithmetic on GPU).
- In the OpenCL implementation, the `float8` accumulator stores X_P, XX_P and Ω_P in double-word precision, the count and $\Omega_P^{(2)}$ in single precision. In the `azimuthal` model, the low word of XX_P is the natural slot for D_P^X.

4.5 Scope of the exact merge: pixel splitting

Equations (12) and (13) are exact for the union of the contributions of A and B. They reproduce a direct integration of $A \cup B$ only if no pixel contributes to both partitions. This is always the case when merging different frames or scan points, since each (pixel, frame) pair belongs to a single frame. It is not the case when merging neighbouring bins of the same frame (azimuthal sectors into a 1D profile, 2D into 1D rebinning) if pixel splitting is enabled.

A pixel i split between A and B with the coefficients c_{iA} and c_{iB} appears in the merge as two contributions with the same x_i. Ω_P and X_P are linear in c and remain exact. But $\Omega_P^{(2)}$, XX_P and D_P^X receive $(c_{iA}^2 + c_{iB}^2)\,n_i^2$ instead of $(c_{iA} + c_{iB})^2\,n_i^2$. As $c_{iA}^2 + c_{iB}^2 < (c_{iA} + c_{iB})^2$, they

are underestimated, and so is the propagated uncertainty. Moreover, pixel splitting correlates neighbouring bins [5], which biases the χ^2 of Equation (21) when the split pixels are shared between azimuthal sectors.

The merge is therefore rigorously exact only when pixel splitting is disabled (pyFAI method `("no", "csr", ...)`), or when the merged partitions do not share any pixel. With hybrid photon-counting detectors such as Eiger or Pilatus, whose point-spread function is about one pixel with no correlation between neighbouring pixels, pixel splitting brings little and is best disabled for data meant to be merged, with bins at least as wide as a pixel. With splitting enabled, the error remains small when the bins are much wider than a pixel, since only the pixels on the bin edges are shared.

5 Consequences for pyFAI

- Every engine implementing the `azimuthal` error model (Cython CSR, CSC, LUT, histogram; OpenCL CSR) should accumulate D_P^X and use Equations (12), (14) or (18), which are exact.
- Integrated results should expose D_P^X next to XX_P (`sum_variance`) and $\Omega_P^{(2)}$ (`sum_normalization2`). Already-integrated results can then be merged exactly with the `azimuthal` error model: union of results, multi-geometry, and rebinning of 2D results into 1D (the latter only without pixel splitting, Section 4.5).
- A numerical validation is provided in the tutorial notebook `doc/source/usage/tutorial/Variance/uncertainties.ipynb`.

6 Validation by simulation

6.1 Setup

Both workflows of Section 1 are compared on a single bin of 1000 pixels, with a normalization n_i uniformly distributed in $[0.7, 1.3]$ and $\omega_i = n_i$. A series is made of $N = 32$ independent frames. Each frame has a background of 20 counts and very "spotty" diffraction: each pixel has a 1 % probability to receive a spot of 2000 counts, at positions drawn again for every frame. Poisson noise is added to the expected signal $n_i(\text{background} + \text{spots})$.

- Workflow 1: the raw frames are averaged, then integrated once with the `azimuthal` model.
- Workflow 2: the union of all (pixel, frame) pairs is integrated, which is exactly what the merge of the per-frame accumulators gives.

The experiment is repeated 1000 times. The true standard error of the mean is the dispersion of the integrated intensity over the repetitions; with 1000 repetitions it is known to about ±2 %.

Applying the decomposition of Section 4.2 to the N values of each pixel gives

$$XX_{[2]} = N\,XX_{[1]} + \sum_i \omega_i^2 \sum_{k=1}^{N} \left(x_{ik} - \bar{x}_i\right)^2 , \tag{19}$$

where $\bar{x}_i$ is the average of pixel i over the frames. Workflow 1 removes the temporal dispersion of each pixel (the last term) before integration, workflow 2 keeps it. With $\Omega_{[2]} = N\,\Omega_{[1]}$ (indices [1] and [2] refer to the workflows), both give the same intensity, but their standard errors differ in general. Writing each pixel value as a true value, plus a systematic azimuthal structure of

variance S (texture, constant over the frames), plus a noise of variance σ^2 independent from frame to frame, one expects

$$\mathrm{sem}_1^2 \approx \frac{\Omega^{(2)}}{\Omega^2}\left(\frac{\sigma^2}{N} + S\right), \qquad \mathrm{sem}_2^2 \approx \frac{\Omega^{(2)}}{\Omega^2}\left(\frac{\sigma^2}{N} + \frac{S}{N}\right). \tag{20}$$

Without texture ($S = 0$), both are equivalent. Spots appearing at random positions in every frame belong to σ^2, not to S.

6.2 Results

Table 3 gives the results. Equation (19) is verified to 2×10^{-16}, and the intensities of both workflows agree to 7×10^{-15}.

	No texture	Fixed texture
True standard error of the mean	1.150	1.182
Workflow 1 (average frames, then integrate)	1.128 ± 0.041	1.211 ± 0.043
Workflow 2 (merge accumulators)	1.129 ± 0.034	1.131 ± 0.032

Table 3: Standard error of the mean estimated by both workflows ($N = 32$ frames, 1000 pixels, 1000 repetitions). The $\pm$ is the dispersion of the estimator over the repetitions.

Without texture. Both workflows give the same standard error, in agreement with the true value: both are about 2 % low, at the limit of the precision of the simulation, which may come from the heavy-tailed distribution of the spotty signal. Workflow 2 is even slightly more stable (± 0.034 instead of ± 0.041), as the dispersion is estimated from 32 000 samples instead of 1000.

With a fixed texture. To show the difference, a systematic structure is added: a Gaussian random offset of standard deviation 15 counts per pixel, drawn once per repetition and kept for its N frames. As expected, workflow 1 keeps the term S and slightly overestimates the error (1.211), whereas workflow 2 divides it by N and underestimates it (1.131). It counts N measurements of the same pixels as independent samples.

6.3 Checking for texture before merging

The absence of texture can be checked on the pre-integrated data themselves, frame by frame, using the azimuthal bins. Without texture, the intensities I_j of the M azimuthal sectors of a radial bin agree within their standard errors sem_j, and

$$\chi^2 = \sum_{j=1}^{M} \frac{(I_j - I)^2}{\mathrm{sem}_j^2} \tag{21}$$

is close to its expectation of $M - 1$ (7 for 8 sectors), where I is the intensity of the merged sectors. The same test can be applied to the merge of several frames, where a systematic texture accumulates whereas random spots average out. Once the absence of texture is validated, the azimuthal bins are merged with Equations (12) and (13), which gives a single 1D integrated signal with an exact propagation of the `azimuthal` error, provided pixel splitting is disabled (Section 4.5). This test only detects a texture at the scale of the azimuthal sectors. A texture at the scale of the pixels, as in the simulation above, cannot be seen in the accumulators; it requires the raw frames, e.g. through the ratio of both terms of Equation (19).

7 Experimental examples

7.1 Spottiness: is the sample exposed enough?

Rietveld refinement assumes an ideal powder: each Debye–Scherrer ring is sampled by a large number of randomly oriented crystallites, and its intensity is uniform along the azimuth. With a small beam, large crystallites or a short exposure, rings break into spots and the integrated intensities become unreliable. The `azimuthal` error model directly measures this: the relative standard error of the mean of radial bin r is

$$\frac{\mathrm{sem}_r}{I_r} = \frac{\sqrt{XX_r}/\Omega_r}{X_r/\Omega_r} = \frac{\sqrt{XX_r}}{X_r}, \tag{22}$$

and pyFAI defines the spottiness index (`IntegrateResult.calc_spottiness`), inspired by [4], as its quadratic mean over the radial bins:

$$S = \sqrt{\frac{\sum_r w_r \, XX_r/X_r^2}{\sum_r w_r}}, \tag{23}$$

with $w_r = 1$ (unweighted) or $w_r = I_r$ (weighted by the intensity of the rings, which emphasizes the Bragg peaks). As a rule of thumb, $S < 0.05$ corresponds to a smooth powder pattern, $0.05 < S < 0.15$ to a mild spottiness or texture, and $S > 0.15$ to a strongly spotty pattern, likely with large grains. S only depends on the accumulators, so it can be calculated after any merge of pre-integrated results.

The dataset used here was recorded on the ID11 beamline of the ESRF on the standard reference material SRM 640 (silicon), with a sub-micron beam at 40 keV and an Eiger2 CdTe 4M detector at 200 Hz. The first data file contains 200 frames, out of 90 files. It is processed in the pyFAI tutorial `doc/source/usage/tutorial/Spotty.ipynb`, with 2000 bins in 2θ and the `azimuthal` error model:

- a single frame is strongly spotty: $S \approx 0.16$ (unweighted) and $S \approx 0.21$ (weighted), with large fluctuations from frame to frame for the weighted index;
- merging the pre-integrated results of successive frames decreases S to about 0.02 after 200 frames. Beyond a few tens of frames, S decreases as $1/\sqrt{N}$, as expected for independent frames (Section 6). The 199 merges take about 0.1 s;
- integrating the average of the 200 raw frames (workflow 1) gives $S = 0.025$ unweighted but $S = 0.061$ weighted. According to Section 6, this difference points to a systematic azimuthal structure on the most intense rings, which survives the averaging. Its origin (residual texture, dynamically masked pixels of the detector, geometry) remains to be investigated.

The spottiness index is therefore a metric to decide when a sample has been exposed long enough: frames are integrated on the fly, their accumulators merged, and the acquisition stops once S goes below a target value, e.g. 0.05 for a Rietveld-quality pattern. For this dataset, about 35 frames are needed, whereas a pure $1/\sqrt{N}$ extrapolation from the first frame would predict 10: the measured curve, not the extrapolation, should be used. Exposing no longer than needed limits the radiation damage to the sample and the cost of the experiment (beam time, storage). The tutorial still uses the approximate merge formula currently implemented in pyFAI; the figures will be updated with the exact merge of Section 4.2.

7.2 Texture and orientation: a Kevlar fibre

Texture is the opposite case: the azimuthal distribution of the intensity is the signal of interest, and the azimuthal sectors must not be merged. The dataset, recorded on the ID13 beamline

of the ESRF, is a tomographic scan of a Kevlar fibre, i.e. poly(*p*-phenylene terephthalamide), a highly oriented polymer [6]. Its main characteristics, read from the data files, are:

- Eiger 4M detector (silicon sensor, 450 μm), 2167 × 2070 pixels, 16-bit frames;
- photon energy 13.45 keV ($\lambda = 0.922$ Å), 10 ms per frame;
- $85 \times 160 = 13\,600$ frames, stored in 7 files of about 13.3 GB in total (compressed);
- a calibration dataset on corundum (α-Al_2O_3), with the corresponding calibration notebook.

The data are available at `http://ftp.edna-site.org/pyFAI/kevlar/`. This well-known dataset has not been processed yet with the method of this note; the following describes the planned analysis.

Integration and storage. Each frame is integrated on 1000 radial × 36 azimuthal bins (sectors of 10°), with the `azimuthal` error model and without pixel splitting, so that the sectors can later be merged exactly (Section 4.5), and the accumulators are stored. With the assumptions of Section 1 (compression ratio of 3 for floating-point data), the 6 accumulators in float32 take about 288 kB per frame, i.e. 3.9 GB for the whole scan, or 2.0 GB when Ω_P, $\Omega_P^{(2)}$ and the count are shared. Compared to the 13.3 GB of raw data, the gain is only 3.4 to 6.8, smaller than in Section 1: the raw frames are 16-bit, and 36 azimuthal sectors are kept.

Texture. For each frame and each radial bin, the χ^2 of Equation (21) with $M = 36$ sectors (expectation 35) tests the isotropy of the signal. Frames where the beam misses the fibre (air scattering only) should be consistent with $\chi^2 \approx 35$, while frames through the fibre should be strongly anisotropic. This validates the texture test on real data, and gives a map of the anisotropy over the scan.

Orientation. For the textured frames, the azimuthal profile I_j of the strong equatorial reflections of Kevlar, (110) and (200) at $d \approx 4.3$ and 3.9 Å [6], gives the orientation of the fibre axis projected on the detector (position of the maxima) and the degree of orientation (width of the peaks, or Hermans orientation factor). The uncertainties sem_j propagated by the `azimuthal` model give error bars on these parameters. Frames with the same orientation can then be merged sector by sector, exactly, to improve the statistics, without going back to the raw data.

References

[1] E. Schubert and M. Gertz. Numerically stable parallel computation of (co-)variance. In *Proceedings of the 30th International Conference on Scientific and Statistical Database Management (SSDBM '18)*, pages 10:1–10:12, Bozen-Bolzano, Italy, 2018. doi:10.1145/3221269.3223036.

[2] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. doi:10.2307/1266577.

[3] E. A. Youngs and E. M. Cramer. Some results relevant to choice of sum and sum-of-product algorithms. *Technometrics*, 13(3):657–665, 1971. doi:10.1080/00401706.1971.10488826.

[4] B. Ingham. Statistical measures of spottiness in diffraction rings. *Journal of Applied Crystallography*, 47(1):166–172, 2014. doi:10.1107/S1600576713029713.

[5] X. Yang, P. Juhás and S. J. L. Billinge. On the estimation of statistical uncertainties on powder diffraction and small-angle scattering data from two-dimensional X-ray detectors. *Journal of Applied Crystallography*, 47(4):1273–1283, 2014. doi:10.1107/S1600576714010516.

[6] M. G. Northolt. X-ray diffraction study of poly(*p*-phenylene terephthalamide) fibres. *European Polymer Journal*, 10(9):799–804, 1974. doi:10.1016/0014-3057(74)90131-1.